스케줄러 서비스

소개

스케줄러 서비스는 Oasis Core 네트워크에서 런타임 위원회 구성과 밸리데이터 선출을 담당하는 핵심 서비스입니다. 이 서비스는 VRF(Verifiable Random Function) 기반의 암호학적 선출 알고리즘을 사용하여 공정하고 예측 불가능한 위원회 구성을 보장합니다.

주요 특징

- **VRF 기반 선출**: 검증 가능한 랜덤 함수를 통한 공정한 위원회 선출
- **스테이크 가중 선출**: 스테이킹된 토큰 양에 비례한 밸리데이터 선출 확률
- **다중 역할 지원**: 워커, 백업 워커 등 다양한 위원회 역할 관리
- **제약 조건 적용**: 엔티티당 최대 노드 수, 최소 풀 크기 등 다양한 제약 조건 지원
- **에포크 기반 교체**: 정기적인 위원회 구성 변경으로 탈중앙화 보장

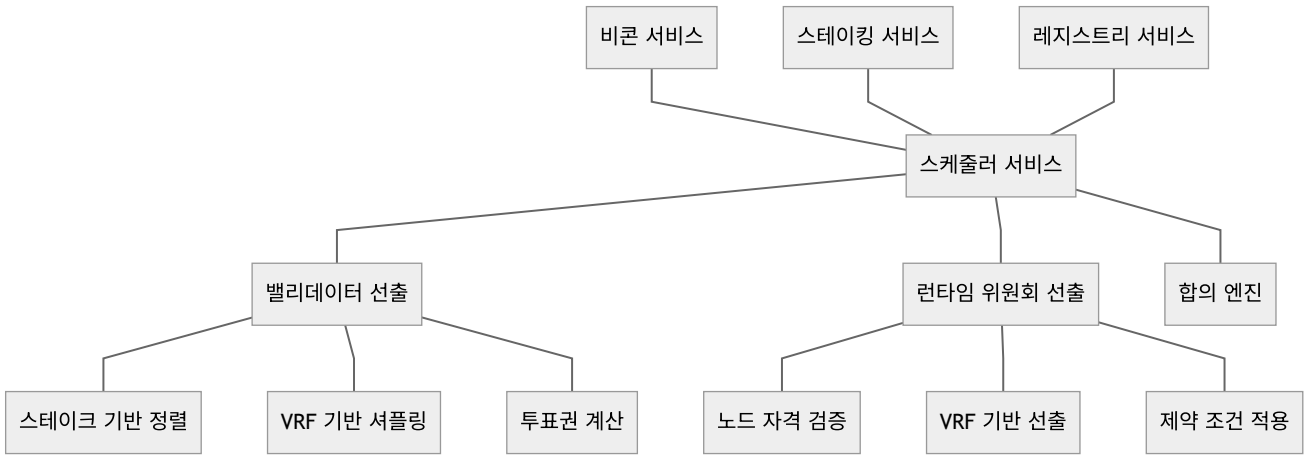
관련 프로세스

- **비콘 서비스**: VRF 증명과 에포크 관리를 통한 선출 엔트로피 제공
- **스테이킹 서비스**: 노드 스테이크 정보 제공 및 선출 자격 검증
- **레지스트리 서비스**: 노드 등록 정보와 런타임 메타데이터 제공
- **합의 프로세스**: 선출된 밸리데이터를 통한 블록 생성과 합의

아키텍처

전체 구조

스케줄러 서비스는 밸리데이터 선출과 런타임 위원회 구성이라는 두 가지 핵심 기능을 제공합니다.



주요 컴포넌트

밸리데이터 선출기

역할: 합의에 참여할 밸리데이터 노드 선출

주요 기능:

- 스테이크 기반 노드 정렬
- VRF 증명을 통한 공정한 셔플링
- 엔티티당 최대 밸리데이터 수 제한
- 투표권 계산 및 할당

위원회 선출기

역할: 런타임별 실행 위원회 구성

주요 기능:

- 런타임 호환성 검증
- 역할별 노드 선출 (워커, 백업 워커)

- TEE 요구사항 검증
- 제약 조건 적용

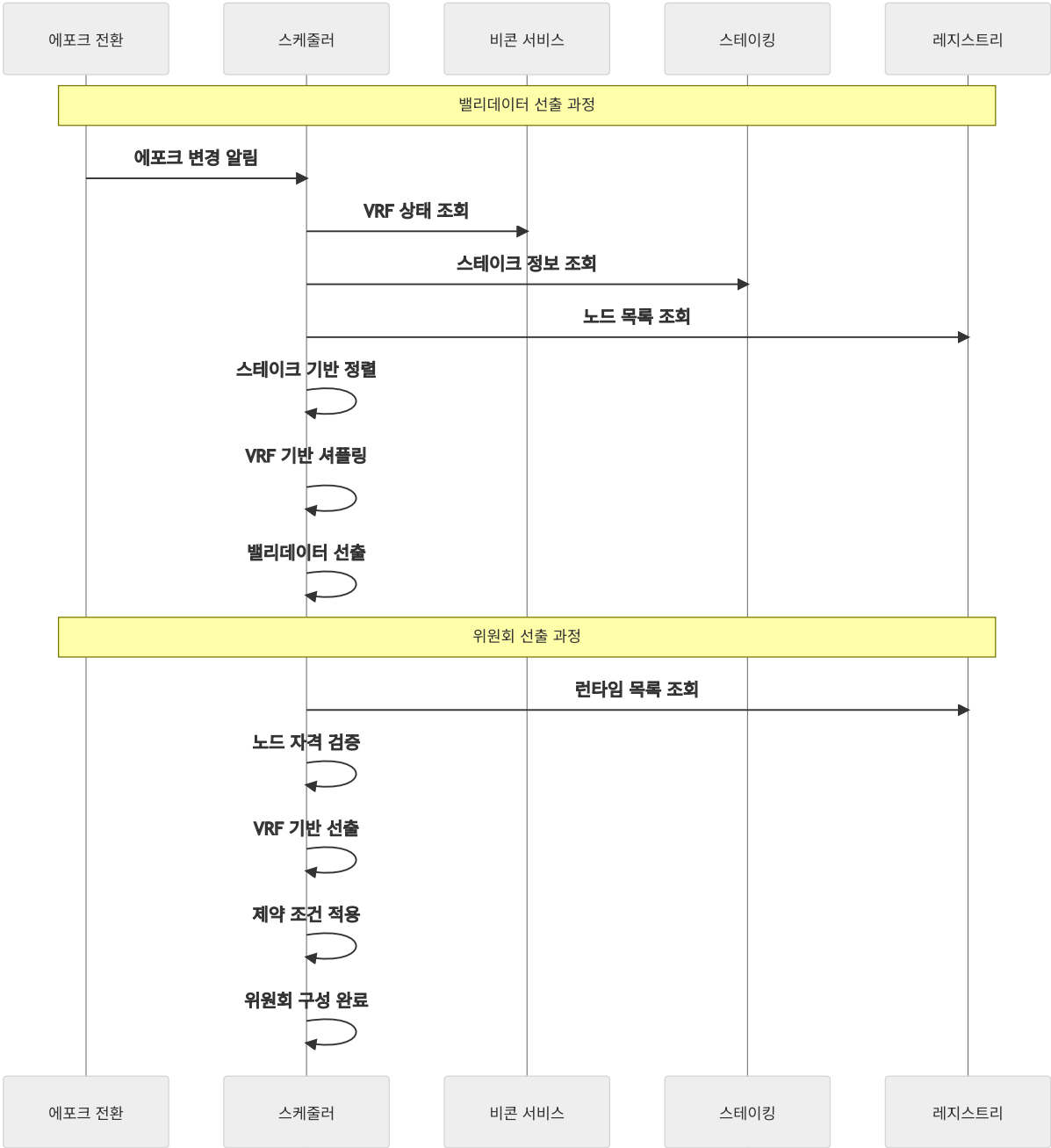
제약 조건 관리자

역할: 선출 과정에서 다양한 제약 조건 적용

주요 기능:

- 엔티티당 최대 노드 수 제한
- 최소 풀 크기 보장
- 밸리데이터 세트 멤버십 요구사항
- 노드 중복 제거

선출 알고리즘


에포크 전환
스케줄러
비콘 서비스
스테이킹
레지스트리
밸리데이터 선출 과정
에포크 변경 알림
VRF 상태 조회
스테이크 정보 조회
노드 목록 조회
스테이크 기반 정렬
VRF 기반 셔플링
밸리데이터 선출
위원회 선출 과정
런타임 목록 조회
노드 자격 검증
VRF 기반 선출
제약 조건 적용
위원회 구성 완료
에포크 전환
스케줄러
비콘 서비스
스테이킹
레지스트리

런타임 위원회 스케줄링 알고리즘

VRF 기반 선출 메커니즘

스케줄러는 VRF(Verifiable Random Function)를 사용하여 예측 불가능하고 검증 가능한 위원회 선출을 수행합니다.

VRF 선출 과정

1. **VRF 증명 수집**: 이전 에포크에서 노드들이 제출한 VRF 증명 수집
2. **베타 값 계산**: 각 노드의 VRF 증명으로부터 베타 값 계산
3. **해시 기반 정렬**: 베타 값을 해시하여 노드들을 정렬
4. **순차적 선출**: 정렬된 순서대로 필요한 수만큼 노드 선출

```go
// VRF 기반 위원회 선출 알고리즘
func electCommitteeByVRF(
prevVRFState *beacon.PrevVRFState,
chainContext []byte,
epoch beacon.EpochTime,
runtimeID common.Namespace,
kind scheduler.CommitteeKind,
role scheduler.Role,
nodeList []*node.Node,
wantedNodes int,
) ([]*scheduler.CommitteeNode, error) {
// 베타 해시 생성기 초기화
```

```go
baseHasher := newCommitteeBetaHasher(
chainContext,
epoch,
runtimeID,
kind,
role,
)
// VRF 증명을 통한 노드 정렬
sortedNodes := sortNodesByHashedBeta(
prevVRFState,
baseHasher,
nodeList,
)
// 필요한 수만큼 노드 선출
var elected []*scheduler.CommitteeNode
for i := 0; i < wantedNodes && i < len(sortedNodes); i++ {
elected = append(elected, &scheduler.CommitteeNode{
Role: role,
PublicKey: sortedNodes[i].ID,
})
}
return elected, nil
```

```
}
```

엔트로피 기반 폴백 메커니즘

VRF 증명이 충분하지 않은 경우, 비콘 엔트로피를 사용한 폴백 메커니즘이 작동합니다.

```
// 엔트로피 기반 선출 (폴백)
func electCommitteeByEntropy(
entropy []byte,
runtimeID common.Namespace,
rngContext []byte,
nodeList []*node.Node,
wantedNodes int,
) ([]*scheduler.CommitteeNode, error) {
// DRBG를 통한 순열 생성
perm, err := GetPerm(entropy, runtimeID, rngContext, len(nodeList))
if err != nil {
return nil, err
}
// 순열에 따라 노드 선출
var elected []*scheduler.CommitteeNode
for i := 0; i < wantedNodes; i++ {
elected = append(elected, &scheduler.CommitteeNode{
Role: scheduler.RoleWorker,
```

```
PublicKey: nodeList[perm[i]].ID,
})
}
return  elected, nil
}
```

노드 선택과 역할 할당 메커니즘

노드 자격 검증

위원회 선출 전에 각 노드의 자격을 엄격하게 검증합니다.

기본 자격 요건

1. **노드 상태 검증**

- 동결되지 않은 상태
- 만료되지 않은 등록
- 충분한 스테이크 보유

2. **런타임 호환성 검증**

- 지원하는 런타임 버전 일치
- 필요한 TEE 하드웨어 지원
- TEE 증명 유효성 검증

3. **VRF 참여 검증**

- VRF 증명 제출 여부 확인
- 증명 유효성 검증

```go
// 실행 워커 노드 자격 검증
func (app *Application) isSuitableExecutorWorker(
ctx *api.Context,
n *nodeWithStatus,
rt *registry.Runtime,
epoch beacon.EpochTime,
registryParams *registry.ConsensusParameters,
) bool {
// 컴퓨트 워커 역할 확인
if !n.node.HasRoles(node.RoleComputeWorker) {
return false
}
// 활성 배포 확인
activeDeployment := rt.ActiveDeployment(epoch)
if activeDeployment == nil {
return false
}
// 런타임 버전 호환성 확인
for _, nrt := range n.node.Runtimes {
if !nrt.ID.Equal(&rt.ID) {
continue
}
```

```
if nrt.Version.ToU64() != activeDeployment.Version.ToU64() {
continue
}
// 노드 정지 상태 확인
if n.status.IsSuspended(rt.ID, epoch) {
return false
}
// TEE 요구사항 검증
switch rt.TEEHardware {
case node.TEEHardwareInvalid:
return nrt.Capabilities.TEE == nil
default:
if nrt.Capabilities.TEE == nil {
return false
}
if nrt.Capabilities.TEE.Hardware != rt.TEEHardware {
return false
}
// TEE 증명 검증
return nrt.Capabilities.TEE.Verify(
registryParams.TEEFeatures,
ctx.Now(),
uint64(ctx.BlockHeight()),
```

```
activeDeployment.TEE,
n.node.ID,
) == nil
}
}
return false
}
```

역할별 선출 과정

워커 노드 선출

워커 노드는 실제 트랜잭션 실행을 담당하는 핵심 역할입니다.

```
// 워커 노드 선출 과정
func electWorkers(
ctx *api.Context,
runtime *registry.Runtime,
eligibleNodes []*node.Node,
groupSize int,
) ([]*scheduler.CommitteeNode, error) {
// VRF 기반 선출
elected, err := electCommitteeByVRF(
prevVRFState,
chainContext,
```

```
epoch,
runtime.ID,
scheduler.KindComputeExecutor,
scheduler.RoleWorker,
eligibleNodes,
groupSize,
)
if err != nil {
return nil, err
}
return elected, nil
}
```

백업 워커 선출

백업 워커는 워커 노드 장애 시 대체 역할을 수행합니다.

```
// 백업 워커 선출 과정
func electBackupWorkers(
ctx *api.Context,
runtime *registry.Runtime,
eligibleNodes []*node.Node,
backupSize int,
) ([]*scheduler.CommitteeNode, error) {
// 워커와 다른 RNG 컨텍스트 사용
```

```
elected, err := electCommitteeByVRF(
prevVRFState,
chainContext,
epoch,
runtime.ID,
scheduler.KindComputeExecutor,
scheduler.RoleBackupWorker,
eligibleNodes,
backupSize,
)
if err != nil {
return nil, err
}
return elected, nil
}
```

제약 조건 적용

엔티티당 최대 노드 수 제한

동일한 엔티티가 위원회를 독점하는 것을 방지합니다.

```
// 엔티티 중복 제거
func dedupEntityNodes(
nodeList []*node.Node,
```

```go
perEntityLimit uint16,
) []*node.Node {
nodesPerEntity := make(map[signature.PublicKey]int)
dedupedNodes := make([]*node.Node, 0, len(nodeList))
for _, node := range nodeList {
if nodesPerEntity[node.EntityID] >= int(perEntityLimit) {
continue
}
nodesPerEntity[node.EntityID]++
dedupedNodes = append(dedupedNodes, node)
}
return dedupedNodes
}
```

최소 풀 크기 보장

위원회 구성에 필요한 최소 노드 수를 보장합니다.

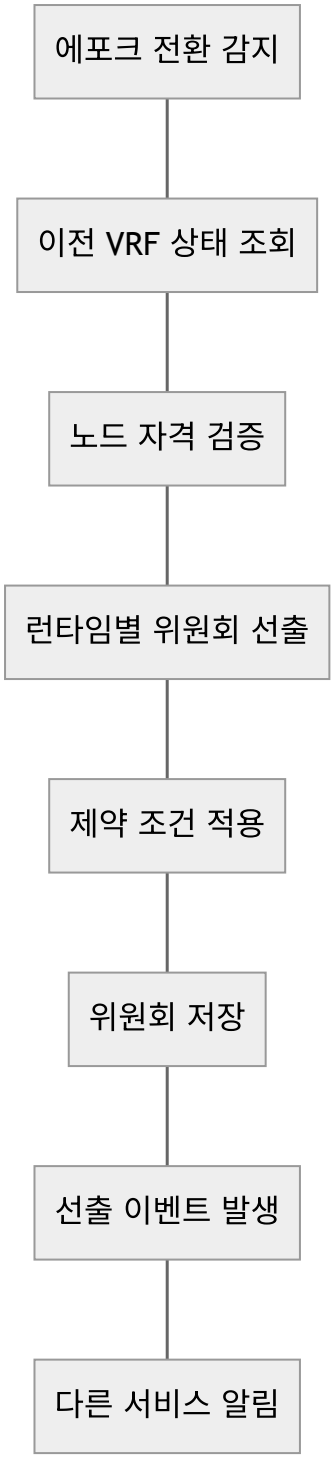
```go
// 최소 풀 크기 검증
func validateMinPoolSize(
eligibleNodes []*node.Node,
minPoolSize int,
wantedNodes int,
) error {
```

```go
if len(eligibleNodes) < minPoolSize {
return fmt.Errorf("insufficient eligible nodes: %d < %d",
len(eligibleNodes), minPoolSize)
}
if len(eligibleNodes) < wantedNodes {
return fmt.Errorf("insufficient nodes for committee: %d < %d",
len(eligibleNodes), wantedNodes)
}
return nil
}
```

위원회 구성과 교체 프로세스

에포크 기반 위원회 교체

위원회는 에포크 전환 시점에 새로 구성됩니다.



위원회 구성 과정

```
// 위원회 구성 메인 로직
func (app *Application) electCommittee(
ctx *api.Context,
schedulerParameters *scheduler.ConsensusParameters,
```

```go
beaconState *beaconState.MutableState,
beaconParameters *beacon.ConsensusParameters,
registryParameters *registry.ConsensusParameters,
stakeAcc *stakingState.StakeAccumulatorCache,
entitiesEligibleForReward  map[staking.Address]bool,
validatorEntities  map[staking.Address]bool,
rt *registry.Runtime,
nodeList []*nodeWithStatus,
kind scheduler.CommitteeKind,
) error {
// 에포크 정보 조회
epoch, _, err := beaconState.GetEpoch(ctx)
if  err != nil {
return  fmt.Errorf("failed to query current epoch: %w", err)
}
// VRF 상태 조회
useVRF := beaconParameters.Backend == beacon.BackendVRF
var  prevState *beacon.PrevVRFState
if  useVRF {
prevState, err = getPrevVRFState(ctx, beaconState)
if  err != nil {
return  err
}
```

```
// 약한 알파 검증
if !prevState.CanElectCommittees && !schedulerParameters.DebugAllowWeakAlpha {
return  app.dropCommittee(ctx, kind, rt.ID)
}
}
// 위원회 크기 결정
groupSizes := make(map[scheduler.Role]int)
switch  kind {
case  scheduler.KindComputeExecutor:
groupSizes[scheduler.RoleWorker] = int(rt.Executor.GroupSize)
groupSizes[scheduler.RoleBackupWorker] = int(rt.Executor.GroupBackupSize)
}
// 빈 위원회 방지
if  groupSizes[scheduler.RoleWorker] == 0 {
return  app.dropCommittee(ctx, kind, rt.ID)
}
// 노드 자격 검증 및 필터링
eligibleNodes := app.filterEligibleNodes(
ctx, nodeList, rt, epoch, registryParameters,
useVRF, prevState, schedulerParameters,
stakeAcc, entitiesEligibleForReward,
)
// 역할별 위원회 선출
```

```go
var  members []*scheduler.CommitteeNode
for  _, role := range []scheduler.Role{scheduler.RoleWorker, scheduler.RoleBacku
if  groupSizes[role] == 0 {
continue
}
elected, err := app.electRole(
ctx, rt, role, eligibleNodes[role], groupSizes[role],
useVRF, prevState, schedulerParameters,
)
if  err != nil {
return  err
}
members = append(members, elected...)
}
// 위원회 저장
committee := &scheduler.Committee{
Kind: kind,
RuntimeID: rt.ID,
Members: members,
ValidFor: epoch,
}
return  schedulerState.NewMutableState(ctx.State()).PutCommittee(ctx, committee)
```

```
}
```

위원회 스케줄링 순서

선출된 위원회 내에서 트랜잭션 스케줄링 순서를 결정합니다.

```
// 스케줄러 순서 계산
func (c *Committee) SchedulerIdx(round uint64, rank uint64) (int, bool) {
var total uint64
// 워커 노드 수 계산
for _, n := range c.Members {
if n.Role != RoleWorker {
break // 워커는 백업 워커보다 앞에 위치
}
total++
}
if rank >= total {
return 0, false
}
// 라운드 기반 순환 스케줄링
idx := (rank + total - round%total) % total
return int(idx), true
}
```

```go
// 스케줄러 랭크 계산
func (c *Committee) SchedulerRank(round uint64, id signature.PublicKey) (uint64
var (
total uint64
idx uint64
isWorker bool
)
for _, n := range c.Members {
if n.Role != RoleWorker {
break
}
if n.PublicKey == id {
isWorker = true
idx = total
}
total++
}
if !isWorker {
return 0, false
}
rank := (round + idx) % total
return rank, true
```

```
}
```

API 및 인터페이스

스케줄러 백엔드 인터페이스

```
// Backend는 스케줄러 구현을 위한 인터페이스입니다
type Backend interface {
// GetValidators는 지정된 에포크의 밸리데이터 목록을 반환합니다
GetValidators(ctx context.Context, height  int64) ([]*Validator, error)
// GetCommittees는 지정된 런타임의 위원회 목록을 반환합니다
GetCommittees(ctx context.Context, request *GetCommitteesRequest) ([]*Committee,
// WatchCommittees는 위원회 변경 이벤트 스트림을 반환합니다
WatchCommittees(ctx context.Context) (<-chan *Committee, pubsub.ClosableSubscrip
// StateToGenesis는 지정된 높이의 제네시스 상태를 반환합니다
StateToGenesis(ctx context.Context, height  int64) (*Genesis, error)
// ConsensusParameters는 스케줄러 합의 파라미터를 반환합니다
ConsensusParameters(ctx context.Context, height  int64) (*ConsensusParameters, e
}
```

위원회 구조체

```
// Committee는 런타임별 위원회입니다
```

```go
type Committee struct {
// Kind는 위원회 종류입니다
Kind CommitteeKind `json:"kind"`
// Members는 위원회 구성원 목록입니다
// 워커가 백업 워커보다 앞에 위치합니다
Members []*CommitteeNode `json:"members"`
// RuntimeID는 위원회가 담당하는 런타임 ID입니다
RuntimeID common.Namespace `json:"runtime_id"`
// ValidFor는 위원회가 유효한 에포크입니다
ValidFor beacon.EpochTime `json:"valid_for"`
}

// CommitteeNode는 위원회 참여 노드입니다
type CommitteeNode struct {
// Role은 노드의 위원회 내 역할입니다
Role Role `json:"role"`
// PublicKey는 노드의 공개키입니다
PublicKey signature.PublicKey `json:"public_key"`
}

// Validator는 합의 밸리데이터입니다
type Validator struct {
// ID는 밸리데이터 노드 식별자입니다
```

```
ID signature.PublicKey `json:"id"`

// EntityID는 밸리데이터 엔티티 식별자입니다

EntityID signature.PublicKey `json:"entity_id"`

// VotingPower는 밸리데이터의 투표권입니다

VotingPower  int64  `json:"voting_power"`

}
```

gRPC API

스케줄러 서비스 정의

```
service SchedulerService {

// GetValidators는 밸리데이터 목록을 조회합니다

rpc  GetValidators(GetValidatorsRequest) returns (GetValidatorsResponse);

// GetCommittees는 위원회 목록을 조회합니다

rpc  GetCommittees(GetCommitteesRequest) returns (GetCommitteesResponse);

// WatchCommittees는 위원회 변경 이벤트를 스트리밍합니다

rpc  WatchCommittees(WatchCommitteesRequest) returns (stream Committee);

// ConsensusParameters는 합의 파라미터를 조회합니다

rpc  ConsensusParameters(ConsensusParametersRequest) returns (ConsensusParameter

}
```

메시지 타입

```
message GetValidatorsRequest {
int64  height = 1; // 블록 높이 (0은 최신 블록)
}

message GetValidatorsResponse {
repeated  Validator  validators = 1; // 밸리데이터 목록
}

message GetCommitteesRequest {
int64  height = 1; // 블록 높이
bytes  runtime_id = 2; // 런타임 ID
}

message GetCommitteesResponse {
repeated  Committee  committees = 1; // 위원회 목록
}

message Validator {
bytes  id = 1; // 노드 ID
bytes  entity_id = 2; // 엔티티 ID
int64  voting_power = 3; // 투표권
}
```

```
message Committee {
uint32  kind = 1; // 위원회 종류
repeated  CommitteeNode  members = 2; // 구성원 목록
bytes  runtime_id = 3; // 런타임 ID
uint64  valid_for = 4; // 유효 에포크
}

message CommitteeNode {
uint32  role = 1; // 역할
bytes  public_key = 2; // 공개키
}
```

설정 및 구성

스케줄러 설정 구조

```
scheduler:
# 최소 밸리데이터 수
min_validators: 1
# 최대 밸리데이터 수
max_validators: 100
# 엔티티당 최대 밸리데이터 수
```

```yaml
max_validators_per_entity: 1

# 투표권 분배 방식 (linear 또는 sqrt)

voting_power_distribution: "linear"

# 에포크 선출 참여 보상 팩터

reward_factor_epoch_election_any: "1"

# 디버그 옵션들 (개발/테스트 환경에서만 사용)

debug_bypass_stake: false

debug_allow_weak_alpha: false

debug_force_elect: {}
```

주요 설정 옵션

| 옵션 이름 | 타입 | 기본값 | 설명 |

|----------|------|--------|------|

| `min_validators` | int | 1 | 최소 밸리데이터 수 |

| `max_validators` | int | 100 | 최대 밸리데이터 수 |

| `max_validators_per_entity` | int | 1 | 엔티티당 최대 밸리데이터 수 |

| `voting_power_distribution` | string | “linear” | 투표권 분배 방식 |

| `reward_factor_epoch_election_any` | quantity | “1” | 선출 참여 보상 팩터 |

| `debug_bypass_stake` | bool | false | 스테이크 검증 우회 (디버그) |

| `debug_allow_weak_alpha` | bool | false | 약한 VRF 알파 허용 (디버그) |

런타임 제약 조건 설정

```yaml
# 런타임 등록 시 위원회 제약 조건 설정
```

```yaml
runtime:
executor:
group_size: 3  # 워커 노드 수
group_backup_size: 1  # 백업 워커 노드 수
constraints:
executor:
worker:
min_pool_size:
limit: 5  # 최소 후보 노드 수
max_nodes:
limit: 1  # 엔티티당 최대 노드 수
validator_set: {} # 밸리데이터 세트 멤버십 요구
backup_worker:
min_pool_size:
limit: 2
max_nodes:
limit: 1
```

환경 변수

환경 변수	설명	예시
`OASIS_SCHEDULER_MIN_VALIDATORS`	최소 밸리데이터 수	`1`

| OASIS_SCHEDULER_MAX_VALIDATORS | 최대 밸리데이터 수 | 100 |

| OASIS_SCHEDULER_DEBUG_BYPASS_STAKE | 스테이크 검증 우회 | false |

코드 예제

기본 사용법

```go
package main

import (
"context"
"fmt"
"log"
"github.com/oasisprotocol/oasis-core/go/scheduler/api"
consensus "github.com/oasisprotocol/oasis-core/go/consensus/api"
)

func main() {
// 합의 서비스 연결
consensusClient, err := consensus.NewConsensusClient("unix:/tmp/oasis-node-conse
if err != nil {
log.Fatal("합의 클라이언트 생성 실패:", err)
}
// 스케줄러 백엔드 가져오기
```

```go
scheduler := consensusClient.Scheduler()

ctx := context.Background()

// 현재 밸리데이터 목록 조회

validators, err := scheduler.GetValidators(ctx, consensus.HeightLatest)

if err != nil {

log.Fatal("밸리데이터 조회 실패:", err)

}

fmt.Printf("현재 밸리데이터 수: %d\n", len(validators))

for i, validator := range validators {

fmt.Printf("밸리데이터 %d: ID=%s, 투표권=%d\n",

i+1, validator.ID, validator.VotingPower)

}

}
```

위원회 조회

```go
// 특정 런타임의 위원회 조회

func getCommittees(scheduler api.Backend, runtimeID common.Namespace) error {

ctx := context.Background()

// 위원회 조회 요청

request := &api.GetCommitteesRequest{

Height: consensus.HeightLatest,
```

```go
RuntimeID: runtimeID,
}
committees, err := scheduler.GetCommittees(ctx, request)
if err != nil {
return fmt.Errorf("위원회 조회 실패: %w", err)
}
for _, committee := range committees {
fmt.Printf("위원회 종류: %s\n", committee.Kind)
fmt.Printf("런타임 ID: %s\n", committee.RuntimeID)
fmt.Printf("유효 에포크: %d\n", committee.ValidFor)
fmt.Printf("구성원 수: %d\n", len(committee.Members))
for i, member := range committee.Members {
fmt.Printf(" 구성원 %d: 역할=%s, ID=%s\n",
i+1, member.Role, member.PublicKey)
}
}
return nil
}
```

위원회 변경 모니터링

```go
// 위원회 변경 이벤트 모니터링
func watchCommitteeChanges(scheduler api.Backend) error {
```

```go
ctx := context.Background()
// 위원회 이벤트 구독
committeeCh, sub, err := scheduler.WatchCommittees(ctx)
if err != nil {
return fmt.Errorf("위원회 이벤트 구독 실패: %w", err)
}
defer sub.Close()
fmt.Println("위원회 변경을 모니터링합니다...")
for {
select {
case committee := <-committeeCh:
fmt.Printf("위원회 업데이트: %s (에포크 %d)\n",
committee.Kind, committee.ValidFor)
// 위원회 변경 시 수행할 작업
if err := handleCommitteeUpdate(committee); err != nil {
log.Printf("위원회 업데이트 처리 실패: %v", err)
}
case <-ctx.Done():
return ctx.Err()
}
}
}
```

```go
func handleCommitteeUpdate(committee *api.Committee) error {
// 새 위원회 구성에 따른 작업 수행
fmt.Printf("새 위원회 구성원 수: %d\n", len(committee.Members))
// 스케줄링 순서 계산 예시
for round := uint64(0); round < 5; round++ {
scheduler, ok := committee.Scheduler(round, 0)
if ok {
fmt.Printf("라운드 %d 스케줄러: %s\n", round, scheduler.PublicKey)
}
}
return nil
}
```

스케줄링 순서 계산

```go
// 위원회 내 스케줄링 순서 계산
func calculateSchedulingOrder(committee *api.Committee, round uint64) {
fmt.Printf("라운드 %d 스케줄링 순서:\n", round)
// 모든 워커 노드의 스케줄링 랭크 계산
var workers []api.CommitteeNode
for _, member := range committee.Members {
if member.Role == api.RoleWorker {
workers = append(workers, *member)
```

```go
}
}
// 랭크별 정렬
type nodeRank struct {
node *api.CommitteeNode
rank  uint64
}
var  rankedNodes []nodeRank
for  i := range  workers {
rank, ok := committee.SchedulerRank(round, workers[i].PublicKey)
if  ok {
rankedNodes = append(rankedNodes, nodeRank{
node: &workers[i],
rank: rank,
})
}
}
// 랭크 순으로 정렬
sort.Slice(rankedNodes, func(i, j  int) bool {
return  rankedNodes[i].rank < rankedNodes[j].rank
})
// 스케줄링 순서 출력
for  i, nr := range  rankedNodes {
```

```
fmt.Printf(" %d. 노드 %s (랭크 %d)\n",
i+1, nr.node.PublicKey, nr.rank)
}
}
```

CLI 사용 예제

```
# 현재 밸리데이터 목록 조회
oasis-node scheduler show-validators

# 특정 런타임의 위원회 조회
oasis-node scheduler show-committees --runtime-id 8000000000000000000000000

# 스케줄러 파라미터 조회
oasis-node scheduler show-parameters

# 위원회 변경 이벤트 모니터링
oasis-node scheduler watch-committees

# 특정 높이의 위원회 조회
oasis-node scheduler show-committees --height 12345 --runtime-id 800000000
```

모니터링 및 메트릭

주요 메트릭

| 메트릭 이름 | 타입 | 설명 |

|---|---|---|

| `oasis_scheduler_elections_total` | Counter | 총 선출 횟수 |

| `oasis_scheduler_committee_size` | Gauge | 위원회 크기 |

| `oasis_scheduler_validator_count` | Gauge | 밸리데이터 수 |

| `oasis_scheduler_election_duration_seconds` | Histogram | 선출 소요 시간 |

| `oasis_scheduler_eligible_nodes` | Gauge | 선출 자격 노드 수 |

| `oasis_scheduler_vrf_proofs_count` | Gauge | VRF 증명 수 |

헬스 체크

```
# 스케줄러 서비스 상태 확인
curl http://localhost:9001/scheduler/status

# 응답 예시
{
"current_epoch": 1234,
"validator_count": 15,
"committee_count": 3,
"last_election_height": 123450,
"vrf_enabled": true,
"election_method": "vrf"
```

```
}
```

로그 모니터링

```
# 스케줄러 관련 로그 확인
journalctl -u oasis-node -f | grep scheduler

# 선출 관련 로그 확인
journalctl -u oasis-node | grep "finished electing committees"

# VRF 관련 로그 확인
journalctl -u oasis-node | grep "VRF.*election"

# 에러 로그 필터링
journalctl -u oasis-node --since "1 hour ago" | grep ERROR | grep schedule
```

문제 해결

일반적인 문제

문제 1: 위원회 선출 실패

증상: 위원회가 구성되지 않거나 빈 위원회가 생성됨

원인:

- 자격을 갖춘 노드 부족

- VRF 증명 부족
- 제약 조건 위반

해결 방법:

1. 노드 자격 요건 확인

```
oasis-node registry list-nodes --verbose
```

2. VRF 증명 상태 확인

```
oasis-node beacon show-vrf-state
```

3. 런타임 제약 조건 검토

```
oasis-node registry show-runtime --runtime-id <ID>
```

문제 2: 밸리데이터 선출 불균형

증상: 특정 엔티티가 과도하게 많은 밸리데이터를 보유

원인:

- 스테이크 분포 불균형
- 엔티티당 최대 밸리데이터 수 설정 문제
- VRF 셔플링 문제

해결 방법:

1. 스테이크 분포 확인

```
oasis-node staking list-accounts --verbose
```

2. 스케줄러 파라미터 조정

```yaml
scheduler:
max_validators_per_entity: 1
```

3. 투표권 분배 방식 변경

```yaml
scheduler:
voting_power_distribution: "sqrt"
```

문제 3: VRF 기반 선출 실패

증상: VRF 선출이 엔트로피 기반 폴백으로 전환됨

원인:

- VRF 증명 제출 부족
- 약한 VRF 알파 값
- VRF 백엔드 설정 문제

해결 방법:

1. VRF 증명 제출 상태 확인

```
oasis-node beacon show-vrf-state
```

2. 노드 VRF 키 설정 확인

```
oasis-node identity show-vrf-key
```

3. 비콘 파라미터 확인

```
oasis-node beacon show-parameters
```

로그 분석

```
# 선출 과정 로그 분석
grep "finished electing committees" /var/log/oasis/node.log | tail -10

# VRF 선출 로그 확인
grep "shuffling by hashed betas" /var/log/oasis/node.log

# 폴백 선출 로그 확인
grep "shuffling by per-epoch entropy" /var/log/oasis/node.log

# 위원회 드롭 로그 확인
grep "failed to drop committee" /var/log/oasis/node.log
```

성능 튜닝

선출 성능 최적화

```
scheduler:
# 최소 풀 크기 조정으로 선출 속도 향상
min_pool_size: 10
# 엔티티당 노드 수 제한으로 복잡도 감소
```

```
max_validators_per_entity: 1
```

VRF 최적화

```
beacon:
vrf_parameters:
# 증명 제출 윈도우 조정
submission_window: 50
# 증명 지연 최적화
proof_delay: 5
```

참조

관련 문서

- 비콘 서비스
- 스테이킹 서비스
- 레지스트리 서비스
- 합의 프로세스
- 아키텍처 개요

소스 코드

- 스케줄러 API
- 스케줄러 구현

- VRF 기반 선출

외부 리소스

- VRF 표준 (RFC 9381)
- 위원회 기반 합의
- 스테이크 기반 선출

마지막 업데이트: 2024년 1월

작성자: Oasis Core 문서화 팀

검토자: 스케줄러 개발팀